CS 344 **Problem Set 4**
Fall 2021 **Due:** Dec 10, 2021, 11:59 p.m.

Name: (Xiaowei Zhang) **NetID**: (xz561)

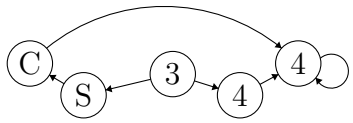
Honor Code: Students may discuss and work on homework problems in groups, which is encouraged. However, each student must write down their solutions independently to show they understand the solution well enough to reconstruct it by themselves. Students should clearly mention the names of the other students who offered discussions. We check all submissions for plagiarism. We take the honor code seriously and expect students to do the same.

Instruction for Submission: This homework has a total of 50 points, it will be rescaled to 10 points as the eventual score. We encourage you to use LaTex to write your answer, because it's particularly suitable to type equations and it's frequently used for writing academic papers. We have provided the homework3.tex file for you, you can write your answer to each question in this .tex file directly after the corresponding question, and then compile the .tex file into a PDF file for submission. As a quick introduction to LaTex, please see this *Learn LaTeX in 30 minutes* [1]. Compiling a .tex file into a PDF file needs installing the LaTex software on your laptop. It's free and open source. But if you don't want to install the software, you can just use this website `https://www.overleaf.com/`, which is also free of charge. You can just create an empty project in this website and upload the homework1.zip, and then the website will compile the PDF for you. You can also directly edit your answers on the website and instantly compile your file. You can also use Microsoft Word or other software if you don't want to use LaTex. If so, please clearly number your answers so that we know which answer corresponds to which question. You also need to save your answers as a PDF file for submission. Finally, please submit your PDF file only. You should name your PDF file as "Firstname-Lastname-NetID.pdf''.

Late Policy: The homework is due on 12/10 (Friday) at 11:59pm. We will release the solutions of the homework on Canvas on 12/14 (Tue) 11:59pm. If your homework is submitted to Canvas before 12/10 11:59pm, there will no late penalty. If you submit to Canvas after 12/10 11:59pm and before 12/14 11:59pm (i.e., before we release the solution), your score will be penalized by 0.9^k, where k is the number of days of late submission. For example, if you submitted on 12/13, and your original score is 80, then your final score will be $80 * 0.9^3 = 58.32$ for $13 - 10 = 3$ days of late submission. If you submit to Canvas after 12/14 11:59pm (i.e., after we release the solution), then you will earn no score for the homework.

Drawing graphs: You might try `http://madebyevan.com/fsm/` which allows you to draw graphs with your mouse and convert it into LaTeX code:

[1] `https://www.overleaf.com/learn/latex/Learn_LaTeX_in_30_minutes`



You can also draw by hand and insert a picture.

Make best use of picture in Latex: If you think some part of your answer is too difficult to type using Latex, you may write that part on paper and scan it as a picture, and then insert that part into Latex as a picture, and finally Latex will compile the picture into the final PDF output. This will make things easier. For instructions of how to insert pictures in Latex, you may refer to the Latex file of our homework 1, which includes several examples of inserting pictures in Latex.

Discussion Group (People with whom you discussed ideas used in your answers if any):

I acknowledge and accept the Honor Code. Please type your initials below:
Signed: (XZ)

---

1. **Longest Common Subsequence** (20 points)

   Consider the two sequences `penguin` and `chicken`, using the LCS algorithm we learnt in class, find the longest common subsequence of these two words.

   (a) (10 points) Work out the 2-dimensional dynamic programming table $T(i, j)$ and provide the length of the LCS.

   [You are expected to work out every element of the dynamic programming table.]

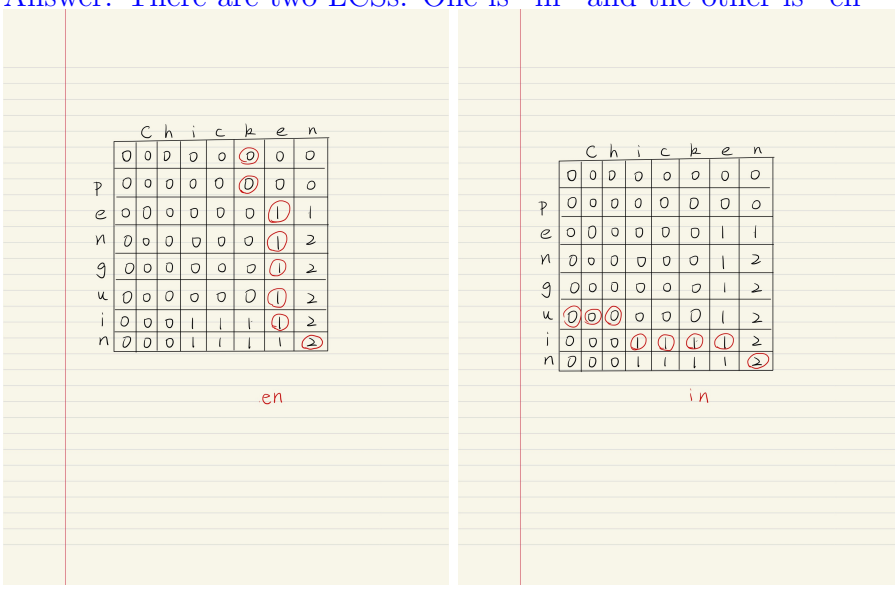
   Answer:

		c	h	i	c	k	e	n
	0	0	0	0	0	0	0	0
p	0	0	0	0	0	0	0	0
e	0	0	0	0	0	0	1	1
n	0	0	0	0	0	0	1	2
g	0	0	0	0	0	0	1	2
u	0	0	0	0	0	0	1	2
i	0	0	0	1	1	1	1	2
n	0	0	0	1	1	1	1	2

   (b) (10 points) Traceback on the table to find out the exact longest common subsequence(s) of the two words.

[You are required to draw the traceback path(s) on the table and find out all the LCSs.]

Answer: There are two LCSs. One is "in" and the other is "en"



2. **Longest Increasing Subsequence** (30 points)

Let A be an array of length n containing real numbers. A *longest increasing subsequence* (LIS) of A is a sequence $0 \leq i_1 < i_2 < \ldots i_\ell < n$ so that $A[i_1] < A[i_2] < \cdots < A[i_\ell]$, so that ℓ is as long as possible. For example, if $A = [6, 3, 2, 5, 6, 4, 8]$, then an LIS is $i_0 = 2, i_1 = 3, i_2 = 4, i_3 = 6$ corresponding to the subsequence $2, 5, 6, 8$. (Notice that a longest increasing subsequence does not need to be unique). In the following parts, we will walk through the recipe that we saw in class for coming up with DP algorithms to develop an $O(n^2)$-time algorithm for finding an LIS.

(a) (10 points) **(Identify optimal sub-structure and a recursive relationship).** We will come up with the sub-problems and recursive relationship for you, although you will have to justify it. Let $D[i]$ be the length of the longest increasing subsequence of $[A[0], \ldots, A[i]]$ that ends on $A[i]$. Expain why

$$D[i] = \max_k \{D[k] + 1 \,:\, 0 \leq k < i, A[k] < A[i]\}.$$

Answer:
when i=0, there is no such k that $0 \leq k < i$, so $D[k] = 0$ and D[i]=D[k]+1 = 1, i.e. just [A[0]]
suppose $0 \leq j < i$ and $A[j] < A[i]$ and we have $D[j] = max_{k_j}\{D[k_j] + 1 : 0 \leq k_j < j, A[k_j] < A[j]\}$, we will certainly have $D[i] = D[j] + 1$ if $D[j]$ is the largest among all $D[k]$ for $0 \leq k < i$ and $A[k] < A[i]$. The resulting sequence will be [sequence of A[j], A[i]]. The max{} formula basically lists all increasing subsequence that ends at some k that fulfills $0 \leq k < i$ and $A[k] < A[i]$ to find out the longest among them and then append A[i] to the sequence. So the result should indeed be the longest increasing subsequence that ends at i.

[**Provide a short informal explanation. It is good practice to write a formal proof, but this is not required for credit.**]

(b) (10 points) **(Develop a DP algorithm to find the value of the optimal solution)** Use the relationship above to design a dynamic programming algorithm that returns the *length* of the longest increasing subsequence. Your algorithm should run in time $O(n^2)$ and should fill in the array D defined above.

[**Provide the pseudo-code, and a brief justification of why the complexity is $O(n^2)$, no formal proof is required.**]

Answer:

```
longest_increasing_sequence_length(D):
    l = D.length()
    \\L store the length of all longest subsequence that ends at i, i from 0
    \\ to l-1(inclusive)
    L = {}
    for j from 0 to l-1:
        L[j] = 1
    longest_length = 1
    for i from 1 to l-1:
        for k from 0 to i-1:
            if D[k] < D[i]:
                L[i] = max{L[k]+1, L[i]}
                longest_length = max{L[i], longest_length}
    return longest_length
```

The time complexity for j loop is O(n) and for i,k loop is at O(n^2) at worst when $i = l - 1$ so k loops from 0 to l-2, overall the time complexity is O(n^2)

(c) (10 points) **(Adapt the DP algorithm to return the optimal solution)** Adapt your algorithm above by tracking some useful information during the DP procedure, so that it returns the actual LIS, not just its length.

[**Pseudocode and a short explanation.**]

Note: Actually, there is an $O(n \log n)$-time algorithm to find an LIS, which is faster than the DP solution in this exercise. Answer:

```
longest_increasing_sequence(D):
    l = D.length()
    \\S store all longest subsequence that ends at i, i from 0
    \\ to l-1(inclusive)
    S = {}
    for j from 0 to l-1:
        S[j] = {D[j]}
    longest_sequence = {}
    longest_length = 1
    for i from 1 to l-1:
        for k from 0 to i-1:
```

```
            if D[k] < D[i] and len(S[k])+1 > len(S[i]):
                S[i] = copy(S[k])
                S[i].append(D[i])
                if len(S[i]) > len(longest_sequence):
                    longest_sequence = S[i]
                    longest_length = len(S[i])
    return longest_sequence, longest_length
```

Instead of having L store all the maximum length of all increasing subsequence that end at i, we now use S to store all the subsequence instead. So the longest sequence will be found along the way just like the largest length. Mind that there might be more than one LIS.